

Redis：实现 Agent 短期记忆存储的最佳方案 已付费

原创 神说要有光 神光的幸福生活 2026年6月1日 12:41 山东

用户和 Agent 聊了几轮之后，对话上下文该存放在哪里呢？

CLI 版的单机 Agent 很简单：

所有对话、截断、摘要都直接放在进程内存里，不需要任何外部存储。

但后端服务不一样：

线上服务一般会部署多个服务实例做负载均衡，单实例内存无法共享会话数据；同时对话属于高频读写的热数据，对响应延迟要求极高，还需要实现会话闲置自动失效。

这种场景下，最近几轮对话、动态截断、历史摘要，这些运行时上下文必须放在 Redis 里。

依靠 Redis 低延迟读写、天然支持 TTL 过期、多实例数据共享的特性，完美支撑短期会话运行。

Redis 只负责承载运行时短期记忆，不会长期保存数据。

每一条消息本身，最终都会异步写入 PostgreSQL 保存，作为永久的聊天记录和可语义检索的长期记忆。

写消息的流程是这样的：

- 用户发消息
- 写入 Redis（更新短期记忆）
- 同时写入 PostgreSQL（落库永久保存）
- 截断、摘要都在 Redis 里做
- 长期记忆检索从 PostgreSQL + 向量查

我们先学下 Redis，然后基于它实现短期记忆。

Redis 是一个高性能的**键值型内存数据库**，也是后端开发中最常用的缓存中间件 之一。

它的数据默认都存在内存里，读写延迟通常在亚毫秒级，天生就适合处理像对话上下文这样的高频读写场景。

它的几个特性，刚好完美适配 Agent 短期记忆的需求：

- **支持 TTL（过期时间）**

会话闲置一段时间后，数据会自动过期清理，不用我们写额外的定时清理逻辑，非常省心。

- **多实例共享数据**

所有 Agent 服务实例都能连接同一个 Redis，用户的会话上下文在任何实例上都能被读取，完美解决了负载均衡下的会话共享问题。

- **丰富的数据结构**

我们可以用 List 直接存对话列表，用 Hash 存会话元信息，用 String 存对话摘要，实现起来非常灵活。

- **支持持久化**

如果需要，Redis 也能通过 RDB/AOF 做数据备份，避免极端情况下会话数据丢失，兼顾性能与可靠性。

接下来，我们就来用一下 Redis

创建项目：

```
mkdir redis-test
cd redis-test
npm init -y
```

创建 docker-compose.yml

```
services:
  # Redis
  redis:
    image:redis:7-alpine
    container_name:agent_redis
    restart:always
    ports:
      -"6379:6379"
    volumes:
      -${DOCKER_VOLUME_DIRECTORY:-.}/volumes/redis:/data
    command:redis-server--appendonlyyes
    healthcheck:
      test:["CMD","redis-cli","ping"]
      interval:5s
      timeout:5s
      retries:5

  # Redis 官方 Web GUI (类似 pgAdmin)
  redisinsight:
    image:redis/redisinsight:2.50
    container_name:redis_insight
    restart:always
    ports:
```

```
    -"5540:5540"

volumes:
  -${DOCKER_VOLUME_DIRECTORY:-.}/volumes/redisinsight:/data

environment:
  -RI_HOST=0.0.0.0

depends_on:
  -redis

networks:
  default:
    name:common-network
```

跑下 redis 以及它的 GUI

我们通过命令创建了 string 类型的 key，并设置了 TTL 过期时间。

redis 还有很多数据类型：

我们过一遍常用的 7 种：

创建 redis-data-types.md

```
# Redis 核心数据类型手册
```

```
## 一、String 字符串
```

适用场景：验证码、Token、登录会话、计数器、分布式锁、配置项、文本类短期记忆

```
**核心命令**
```

```
set key value
```

```
get key
```

```
setex key 秒数 value
```

`set key value nx ex 秒数`

`incr key`

`decr key`

`incrby key 步长`

****真实业务示例****

手机验证码, 5 分钟过期

`setex verification:mobile:13800138000 300 "666888"`

用户登录 Token, 24 小时过期

`setex session:token:adf245kjndsa3 86400 "userid:1001"`

文章阅读量自增

`incr counter:article:1024`

分布式锁, 10 秒过期, 防止重复执行

`set lock:order:2001 "locked" nx ex 10`

AI 对话摘要, 1 小时过期

`setex agent:memory:user:1001 3600 "用户想学习 PostgreSQL 向量检索"`

二、Hash 哈希

适用场景: 用户信息、商品资料、电商购物车、结构化对话上下文

****核心命令****

`hset key field value`

`hmset key field1 value1 field2 value2`

`hget key field`

`hgetall key`

`hkeys key`

`hvals key`

`hincrby key field 增量`

****真实业务示例****

存储用户基础信息

`hset user:info:1001 name "张三" age 28 phone "13800138000"`

电商购物车, 字段为商品 ID, 值为购买数量

`hset cart:user:1001 product:10086 2 product:10087 1`

存储 AI 会话完整上下文

`hset agent:session:user:1001 messages "最近 5 轮对话" summary "对话摘要"`

三、List 列表

适用场景：消息队列、任务队列、操作日志、聊天历史、有序记录

****核心命令****

```
lpush key value1 value2
rpush key value1 value2
lrange key 0 -1
lpop key
rpop key
llen key
```

****真实业务示例****

订单消息队列，右侧入队

```
rpush queue:order "order_1001""order_1002"
```

用户浏览历史，左侧插入最新记录

```
lpush user:history:1001 "查看了 AI 课程""查看了 Redis 教程"
```

后台任务队列

```
rpush queue:task "生成对话摘要""向量入库"
```

四、Set 集合

适用场景：数据去重、每日签到、IP 黑名单、共同好友、权限标签

****核心命令****

```
sadd key value1 value2
smembers key
sismember key value
sinter key1 key2
sunion key1 key2
sdiff key1 key2
```

****真实业务示例****

记录当日签到用户

```
sadd sign:20250820:user 1001 1002 1003
```

网站 IP 黑名单

```
sadd blacklist:ip "192.168.1.100""192.168.1.101"
```

查询两位用户的共同好友

```
sinter user:friend:1001 user:friend:1002
```

五、ZSet 有序集合

适用场景：各类排行榜、内容热度排序、用户积分排名、权重队列

****核心命令****

```
zadd key score member
zrange key 0 -1
zrevrange key 0 -1
zscore key member
zrank key member
```

****真实业务示例****

课程热度排行榜，数值为热度分数

```
zadd rank:course 98 "PostgreSQL 实战" 95 "AI Agent 开发" 92 "Redis 从入门到精通"
```

用户积分排行榜

```
zadd rank:user:points 1000 "张三" 850 "李四"
```

文章热度排序

```
zadd hot:article 1200 "article:1024" 980 "article:1025"
```

六、Bitmap 位图

适用场景：海量用户签到记录、在线状态统计、布尔型数据存储，极致节省内存

****核心命令****

```
setbit key 偏移量 0/1
getbit key 偏移量
bitcount key
```

****真实业务示例****

记录用户当月签到，第 5 天、第 10 天完成签到

```
setbit user:sign:1001:202508 5 1
```

```
setbit user:sign:1001:202508 10 1
```

统计该用户当月总签到天数

```
bitcount user:sign:1001:202508
```

七、Geo 地理位置

适用场景：附近门店、附近的人、两地距离计算、位置检索

****核心命令****


```
geoadd key 经度 纬度 名称
geodist key 名称1 名称2 km
```

****真实业务示例****

添加线下门店经纬度信息

```
geoadd shop:location 116.481028 39.921983 "北京总店"
```

计算两家门店之间的直线距离，单位千米

```
geodist shop:location "北京总店" "上海分店" km
```

数据类型场景速查表

数据类型	典型业务场景
----	----
String	验证码、Token、计数器、分布式锁、文本记忆
Hash	用户信息、商品数据、购物车、结构化会话
List	消息队列、任务队列、浏览/聊天历史
Set	签到、数据去重、黑名单、好友关系
ZSet	排行榜、热度排序、积分排名
Bitmap	批量签到、海量布尔状态统计
Geo	位置检索、距离计算、附近门店/人群

试一下：

神光的编程秘籍

这样我们过了一遍 7 种常用 Redis 数据类型，以及各自适合的业务场景。

那在代码里咋用 redis 呢？

安装依赖：

```
pnpm install ioredis
```

创建 src/redis-test.mjs

```
import Redis from 'ioredis';

// 创建 Redis 客户端
const redis = new Redis({
  host: 'localhost',
  port: 6379,
  db: 0,
});

// 监听连接
redis.on('connect', () => {
  console.log('✅ ioredis 连接成功 (mjs 版) ');
});

// 错误监听
redis.on('error', (err) => {
  console.error('❌ Redis 连接失败: ', err);
});

// 执行操作
asyncfunction runRedisDemo() {
  try {
    // =====
    // 1. String 字符串
    // =====
    await redis.set('name', '张三');
    await redis.set('code', '6666', 'EX', 300); // 5 分钟过期
    console.log('String name:', await redis.get('name'));

    // =====
    // 2. Hash 哈希
    // =====
    await redis.hset('user:1001', 'name', '李四', 'age', 28);
    console.log('Hash user:', await redis.hgetall('user:1001'));

    // =====
    // 3. List 列表
```

```

// =====
await redis.lpush('task:list', '任务1', '任务2');
await redis.rpush('task:list', '任务3');
console.log('List:', await redis.lrange('task:list', 0, -1));

// =====
// 4. Set 集合
// =====
await redis.sadd('tag:set', 'redis', 'nest', 'node');
console.log('Set:', await redis.smembers('tag:set'));

// =====
// 5. ZSet 有序集合
// =====
await redis.zadd('score:rank', 99, '小明', 95, '小红');
console.log('ZSet 排名:', await redis.zrange('score:rank', 0, -1));

// =====
// 6. 分布式锁 (标准写法)
// =====
const lockKey = 'lock:order:1001';
const lockResult = await redis.set(lockKey, 'locked', 'NX', 'EX', 10);
console.log('分布式锁:', lockResult ? '加锁成功' : '加锁失败');

} catch (err) {
  console.error('执行异常: ', err);
}
}

// 运行
runRedisDemo();

```

跑一下:

redis 的数据结构、应用场景，代码里怎么操作都会了。

接下来我们可以实现刚开始的需求：

基于 redis 实现 agent 短期记忆的存储。

安装 langgraph 和 deepagents：

```
pnpm install @langchain/langgraph @langchain/openai deepagents dotenv langchain zod
```

创建 src/agent-with-redis-memory.mjs

```
/**
 * 基于 Redis 的 Agent 短期记忆
 *
 * 模式：
 * - invoke 前：从 Redis 读取该会话的 messages
 * - invoke 后：把 agent 返回的 messages 写回 Redis (带 TTL)
 * - 压缩：由 langchain summarizationMiddleware 在 agent 内部完成
 *
 * 前置：docker compose up -d redis
 *
 * 运行：node src/agent-with-redis-memory.mjs
```

```

* 输入 exit / quit / :q 退出; :clear 清空当前会话记忆
*/

import "dotenv/config";
import Redis from "ioredis";
import * as readline from "node:readline/promises";
import { stdin, stdout } from "node:process";
import { ChatOpenAI } from "@langchain/openai";
import {
  mapChatMessagesToStoredMessages,
  mapStoredMessagesToChatMessages,
} from "@langchain/core/messages";
import { createAgent, HumanMessage, summarizationMiddleware } from "langchain";

const REDIS_HOST = process.env.REDIS_HOST ?? "localhost";
const REDIS_PORT = Number(process.env.REDIS_PORT ?? 6379);
const REDIS_DB = Number(process.env.REDIS_DB ?? 0);
const MEMORY_TTL = Number(process.env.MEMORY_TTL_SECONDS ?? 1800);
const KEY_PREFIX = process.env.MEMORY_KEY_PREFIX ?? "agent:short_memory";
const SESSION_ID = process.env.MEMORY_SESSION_ID ?? "demo_user_001";

const summaryPrompt = `你是对话摘要助手。请用中文总结以下对话，包含：
1. 讨论的主要话题
2. 用户提到的重要事实（姓名、偏好、日期等，务必保留原文信息）
3. 继续对话所需的关键上下文

保持简洁，不要编造，不要遗漏用户明确说过的信息。

待摘要的对话：
{messages}

摘要：`;

class RedisMessageStore {
  constructor({ redis, keyPrefix, ttlSeconds }) {
    this.redis = redis;
    this.keyPrefix = keyPrefix;
    this.ttlSeconds = ttlSeconds;
  }

  messagesKey(sessionId) {
    return `${this.keyPrefix}:${sessionId}:messages`;
  }

  async loadMessages(sessionId) {
    const raw = await this.redis.get(this.messagesKey(sessionId));
    if (!raw) return [];
    return mapStoredMessagesToChatMessages(JSON.parse(raw));
  }

```

```

    }

    async saveMessages(sessionId, messages) {
        const payload = JSON.stringify(mapChatMessagesToStoredMessages(messages));
        await this.redis.set(this.messagesKey(sessionId), payload, "EX", this.ttlSeconds);
    }

    async clear(sessionId) {
        await this.redis.del(this.messagesKey(sessionId));
    }

    async ttl(sessionId) {
        return this.redis.ttl(this.messagesKey(sessionId));
    }
}

async function invokeWithMemory(agent, store, sessionId, userText) {
    const history = await store.loadMessages(sessionId);
    console.log(`  ↳ 从 Redis 加载 ${history.length} 条历史`);

    const result = await agent.invoke(
        { messages: [...history, new HumanMessage(userText)] },
        { recursionLimit: 30 },
    );

    await store.saveMessages(sessionId, result.messages);
    const ttl = await store.ttl(sessionId);
    console.log(`  ↳ 写回 Redis ${result.messages.length} 条 (TTL ${ttl}s)`);

    return result;
}

const redis = new Redis({ host: REDIS_HOST, port: REDIS_PORT, db: REDIS_DB });

redis.on("connect", () => console.log("✅ Redis 已连接"));
redis.on("error", (err) => console.error("❌ Redis 错误:", err.message));

const store = new RedisMessageStore({
    redis,
    keyPrefix: KEY_PREFIX,
    ttlSeconds: MEMORY_TTL,
});

const model = new ChatOpenAI({
    model: process.env.MODEL_NAME,
    apiKey: process.env.OPENAI_API_KEY,
    configuration: { baseUrl: process.env.OPENAI_BASE_URL },
    temperature: 0,

```

```

});

const agent = createAgent({
  model,
  tools: [],
  systemPrompt:
    "你是会话助手。记住用户提到的关键事实，中文简短回答。若消息中有对话摘要，请据此继续对话。",
  middleware: [
    summarizationMiddleware({
      model,
      summaryPrompt,
      trigger: { messages: 8 },
      keep: { messages: 4 },
    }),
  ],
});

console.log("输入 exit / quit / :q 退出, :clear 清空记忆\n");

const rl = readline.createInterface({ input: stdin, output: stdout });
let prevCount = (await store.loadMessages(SESSION_ID)).length;

try {
  while (true) {
    const userText = (await rl.question("你: ")).trim();
    if (!userText) continue;

    if (["exit", "quit", ":q"].includes(userText.toLowerCase())) break;

    if (userText === ":clear") {
      await store.clear(SESSION_ID);
      prevCount = 0;
      console.log("已清空当前会话记忆\n");
      continue;
    }

    const { messages } = await invokeWithMemory(agent, store, SESSION_ID, userText);
    console.log("\n助手:", messages.at(-1)?.content);
    console.log(`当前消息数: ${messages.length}`);
    if (messages.length < prevCount + 2) {
      console.log("  ✂ 已触发压缩");
    }
    prevCount = messages.length;
    console.log();
  }
} finally {
  rl.close();
}

```

```
await redis.quit();
```

用到的环境变量在 .env 里配置：

```
# Redis (默认与 docker-compose 一致)
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_DB=0

# 短期记忆: key 前缀、会话 ID、过期时间 (秒)
MEMORY_KEY_PREFIX=agent:short_memory
MEMORY_SESSION_ID=demo_user_001
MEMORY_TTL_SECONDS=1800
```

跑一下：

神光的编程秘籍

这样我们就基于 redis 实现了短期记忆的存储，具体的摘要 + 截断的压缩逻辑是用的 deepagents 的。

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

Agent 基本都是用 Redis 做短期记忆的存储，用 PostgreSQL 做历史消息、长期记忆存储。

我们学了 Redis 的 7 种常用数据类型 string、hash、list、set、zset(有序集合)、bitmap (位图)、geo (地理位置)

过了一遍它们的读写命令，以及各自典型的业务场景。

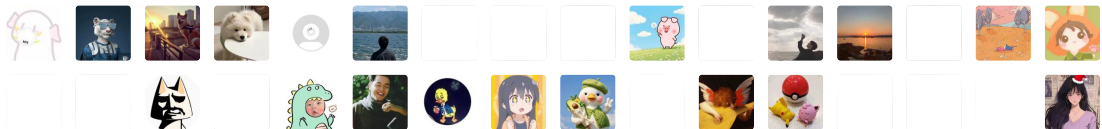
然后基于 Redis + DeepAgents 实现了短期记忆。

DeepAgents 的压缩中间件做消息的摘要 + 截断，Redis 做短期记忆的存储。

一般会把每轮消息存入 PostgreSQL 做历史消息，还能用来基于向量做长期记忆的检索。

Redis 基本是 Agent 做短期记忆存储的最佳方案了。

2774 人付费



转型 Agent 全栈工程师：企业级知识库项目 · 目录

上一篇

PostgreSQL：AI 时代最适合的数据库

下一篇

Mem0：分层记忆 + 三路召回的长期记忆方案

留言 4

写留言



HP 广东 5天前

光哥，上面是基于TTL短期实现吧，上面的实现如果超过ttl时间，短期记忆不就丢了吗



神光的幸福生活 作者 5天前

是的，本来就是临时的，时间长了不聊就会忘记



LDQ 上海 6月3日

可能是快男吧，ai时代我们都需要成为快男



escape 河北 6月1日

太快了，光哥

首评